File IO Lab (CodeGrade)

[(https://github.com/learn-co-curriculum/python-p3-fileio-lab)](https://github.com/learn-co-curriculum/python-p3-fileio-lab) [(https://github.com/learn-co-curriculum/python-p3-fileio-lab/issues/new)](https://github.com/learn-co-curriculum/python-p3-fileio-lab/issues/new)

Learning Goals

- Practice using file IO.

Key Vocab

- **Module**: a file containing Python definitions and statements. A module's functions, classes, and global variables can be accessed by other modules.
- **Package**: a collection of modules that can be accessed as a group using the package name.
- `import` : the Python keyword used to access data from other packages and modules inside of the current module.
- **PyPI**: the **Py**thon **P**ackage **I**ndex. A repository of Python packages that can be downloaded and made available to your application.
- `pip` : the command line tool used to download packages from PyPI. `pip` is installed on your computer automatically when you download Python.
- **Virtual Environment**: a collection of modules, packages, and scripts that can be activated or deactivated at any time.
- **Pipenv**: a virtual environment tool that uses `pip` to manage the modules, packages, and scripts that you intend to use in your application.

Introduction

In the last lesson we learned about the tools that can help us work with files. Creating and manipulating files is a crucial skill that can help you create useful and complex programs. In this lab we will explore file IO.

Instructions

Lets practice! In the file `lib/file_io.py` write a function called `write_file` that takes in the arguments `file_name` and `file_content` .

The `file_name` can be a combined file path/name, you will need to add the `.txt` file extension to the `file_name` when opening a file for all three of the methods.

This function should use `file_name` and `file_content` to write a `.txt` file.

Write a `append_file` function that takes in the same parameters and appends to the `.txt` file.

Write a `read_file` function that takes in a file name and returns the content of the `.txt` file.

Example usage:

```
# use write_file function.
write_file(file_name="logfile", file_content="Log 1: 5 bananas added" )
append_file(file_name="logfile", append_content="Log 2: 3 bananas subtracted")

read_file(file_name="logfile")
# Log 1: 5 bananas added
# Log 2: 3 bananas subtracted
```

Time to get some practice! Write your code in the `file_io.py` file in the `lib/` folder. Run `pytest -x` to check your work.

When all of your tests are passing, submit your work using `git` .

Resources

- [Python File IO](https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files) (https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files)

This tool needs to be loaded in a new browser window

Load File IO Lab (CodeGrade) in a new window